

Public Safety Alerts Aggregator & Explorer

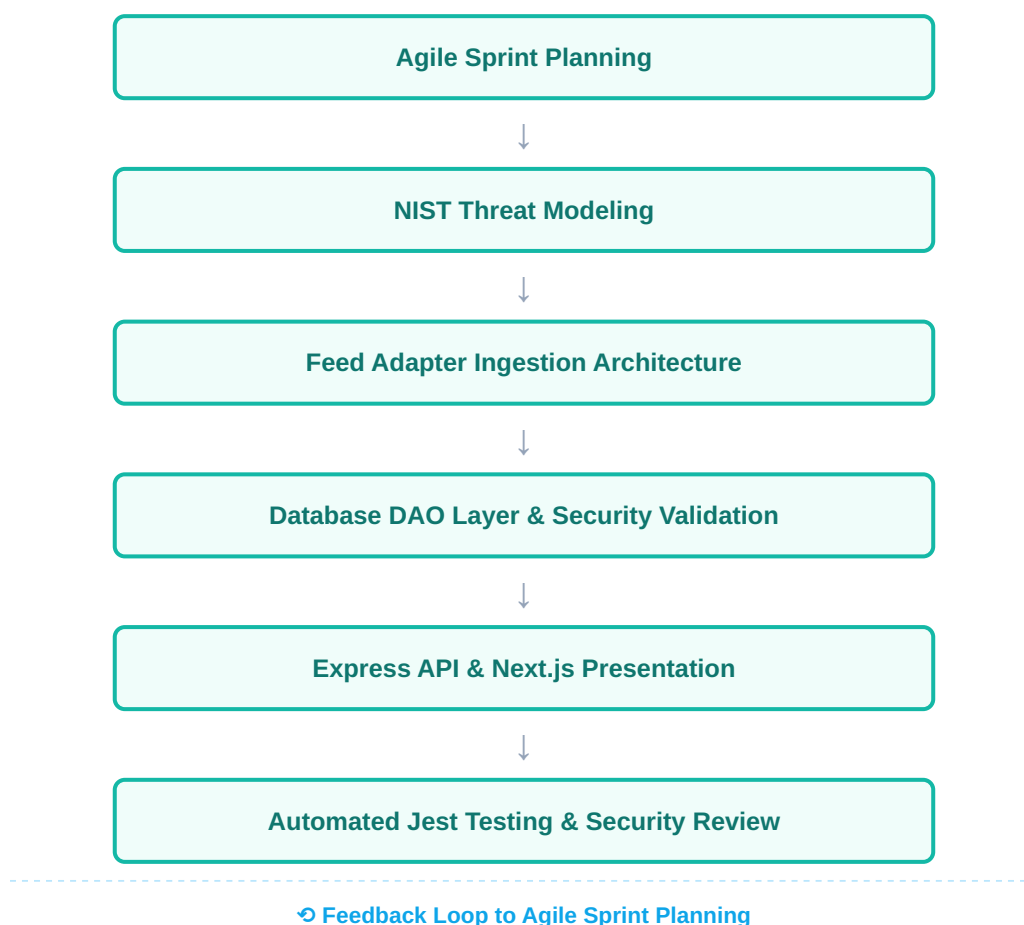
A3: Solution Architecture & Design Walkthrough

TEAM	Critical Signal (Group 2026-16)
MEMBERS	Ettore Vescio, Christian Finocchiaro, Thomas Smith, Pradip Pandey
COURSE	ITC303/ITC306 — Software Development Project 1
ASSESSMENT	A3 — Solution Architecture & Working Prototype Walkthrough

1. APPROACH & METHODOLOGY SELECTION

Our team selected a hybrid delivery approach that merges **Agile Software Development** with a **Risk-Based, Secure-by-Design Framework** (aligned to NIST risk treatment guidelines). This combination ensures that the application is built iteratively to accommodate changing requirements while guaranteeing that security, resilience, and operational reliability are baked into the architecture from day one rather than bolted on as an afterthought.

ALIGNMENT & INFLUENCE OF HYBRID APPROACH



1. Planning & Governance (A1/A3):

- Agile sprints allowed the team to deliver an end-to-end prototype in Session 1, focusing on core pipelines (Ingestion → Storage → Exploration).
- Our NIST-aligned risk assessment forced early identification of trust boundaries—specifically where external, untrusted feeds (NSW RFS and Transport NSW APIs) enter our system. Threat modeling was completed *before* writing database or ingestion code.

2. Requirements & Technology Decisions (A2):

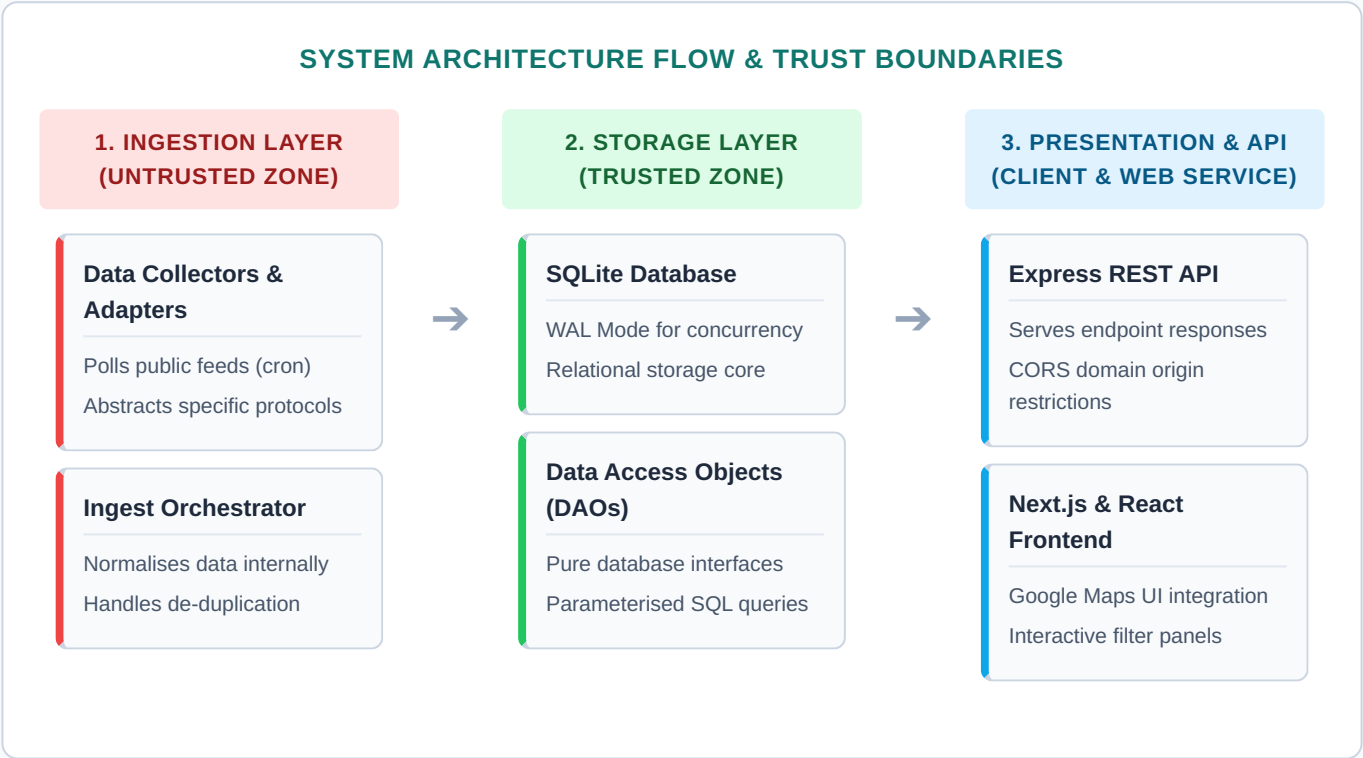
- Instead of treating security as a post-implementation checklist, security requirements (e.g., SQL injection prevention, input sanitisation, and secrets protection) were prioritized in the product backlog as high-priority user stories.
- Technology choices were driven by the need for rapid deployment and visual proof-of-concept (e.g., Next.js and SQLite) while maintaining a strict, MySQL-compatible relational schema that supports clean migration in Session 2.

3. Architecture, Design, & Validation (A3):

- Designed a **Feed Adapter Pattern** to isolate external dependencies. If a public feed goes offline or changes its payload structure, our system catches the error gracefully, logs the event, and continues displaying cached data.
- Built a comprehensive validation strategy that combines automated unit testing (Jest/Supertest) with manual regression testing to verify that no vulnerable data flows from the public feeds to the web frontend.

2. SYSTEM ARCHITECTURE & DIAGRAMS

The *Critical Signal* system uses a classic **3-Tier Architecture** consisting of an Ingestion Layer, a Storage Layer, and a Presentation Layer.



3. KEY DESIGN DECISIONS & TRADE-OFFS

The development of the prototype was shaped by six major architectural decisions. Each decision was evaluated against project constraints, stakeholder requirements, and potential risks.

#	DESIGN DECISION	JUSTIFICATION	TRACED REQUIREMENTS	EXPLICIT TRADE-OFFS
1	SQLite Database for Session 1 (with migration roadmap to MySQL)	Immediate, zero-config relational setup for local demonstration. Eliminates DB admin overhead during rapid prototyping, maintaining 100% SQL schema compatibility.	Client Brief (Session 1): Relational storage, ease of local deployment.	SQLite lacks high concurrent writes. Session 2 will migrate to a MySQL instance using the decoupled DAOs.
2	Data Access Object (DAO) Pattern	Fully decouples the database queries from the business and ingestion logic.	Maintainability: Essential for Session 2 migration from SQLite to MySQL.	Initial setup overhead of writing base DAO interfaces and separate DAOs.
3	Feed Adapter Pattern for Ingestion	Decouples source-specific parsing logic. If an API payload changes, only that specific adapter is quarantined, preventing system-wide crashes.	Client Brief (Session 2): Stability and ease of adding new feeds.	Slight abstraction overhead and duplicate boilerplates for initial setup.
4	Parameterised SQL Queries	Prohibits string concatenation in SQL. User input/external feed variables are passed via bindings, neutralizing SQLi risks.	Security NFR: Prevention of data corruption & malicious execution.	Requires strict discipline in development; direct query execution disabled.
5	@vis.gl/react-google-maps Integration	Provides native React integration for Google Maps, leveraging advanced marker rendering and efficient viewport updates.	Client Brief & A2: High-quality geospatial visualization of alerts.	Requires a valid API Key; billing scales with traffic (mitigated with caching later).
6	Write-Ahead Logging (WAL) Mode	Enforcing WAL mode (<code>PRAGMA journal_mode = WAL</code>) and synchronous = NORMAL on database initialization.	Performance NFR: Concurrent read/write capability.	Slight increase in database file overhead and write checktimes (negligible).

4. NON-FUNCTIONAL REQUIREMENTS (NFRS)

A robust architecture is characterized by its Non-Functional Requirements. Below is the technical specification of the security, performance, reliability, and compliance controls integrated into our prototype.

A. Cybersecurity & Risk Controls (Secure-by-Design)

- **SQL Injection (SQLi) Prevention:** Enforced across all DAOs. Database operations use standard placeholder parameters (`?` or `$param`) to ensure that database engines compile the SQL query structure separately from the data.
- **Cross-Origin Resource Sharing (CORS):** The Express API blocks all unauthorized origins. Our `allowedOrigins.js` middleware restricts incoming requests strictly to the Next.js client URL, neutralizing unauthorized data-scraping and cross-site scripting (XSS) request attempts.
- **XSS Mitigation:** React 19 natively escapes HTML entities by default when rendering JSX variables. For dynamic description fields, the application runs a strict content validation check, sanitizing HTML input to remove any embedded `<script>` or event-handler tags.

- **Secret Management:** Sensitive keys (such as the Google Maps API Key) are isolated in local `.env` files, which are excluded from version control via `.gitignore`.

B. Performance & Scalability

- **Database Concurrency:** The SQLite engine is configured in **WAL mode**, which permits simultaneous read operations even while a background ingestion cycle is active.
- **Geospatial Indexes:** Indexing has been implemented on high-frequency query columns including `alerts.issued_at` and `alerts.category_id` to guarantee sub-millisecond database response times during heavy web traffic.

C. Reliability & Maintainability

- **Fault-Tolerant Ingestion:** If an external feed fails (e.g., DNS error, HTTP 500, network timeout), the `IngestOrchestratorService` catches the exception, logs a detailed error trace to `eventLog.txt`, and terminates the sprint gracefully, leaving the web app functional with cached alerts.
- **Separation of Concerns:** The separation of Data Collection → Services → DAOs → Controllers makes the codebase modular, clean, and highly readable.

D. Compliance & Privacy

- **Australian Privacy Act 1988 Compliance:** To protect student and staff privacy, the system adheres to strict data-minimisation principles. For the notification/subscription subsystem, we do *not* collect physical addresses, names, or phone numbers. The database only stores:
 1. A standard email address.
 2. Hashed and salted preference records containing category and region IDs.

This ensures that a database compromise would expose zero Personally Identifiable Information (PII) or geographic tracking data.

5. TECHNOLOGY STACK & TOOLING JUSTIFICATION

TECHNOLOGY	ROLE	JUSTIFICATION	ALTERNATIVES CONSIDERED & REJECTED
Next.js 16.2 & React 19	Presentation Tier	Enables server-side rendering (SSR) for fast initial loads, robust component-based UI development, and modern React hook integration.	React SPA (Vite): Rejected due to poorer SEO capabilities and lack of built-in file-routing.
Node.js & Express.js	Application Tier	High-performance, asynchronous event-driven backend. JavaScript unification across client and server reduces context-switching.	Python (Flask/Django): Rejected to keep the codebase unified in JavaScript, improving team velocity.
SQLite (better-sqlite3)	Storage Tier	Zero-config, self-contained, and highly efficient. Runs instantly in any local environment, which is perfect for a Session 1 demo.	PostgreSQL/MySQL: Postponed to Session 2 to eliminate complex database administration during local prototype evaluation.
@vis.gl/react-google-maps	Geospatial Rendering	Official, performant React wrapper for the Google Maps JavaScript API, supporting advanced markers and native layout controls.	Leaflet / OpenStreetMap: Rejected due to poorer rendering speeds and complex custom styling requirements.
Jest & Supertest	Testing Suite	Standard testing framework in the JS ecosystem. Supertest enables robust, automated integration testing of HTTP routes.	Mocha / Chai: Jest was preferred for its zero-config setup and built-in code coverage reports.

TIP: REPOSITORY ACCESS

The complete architectural visual layout, ERD, and source files can be inspected in the repository at [public-safety-alerts-aggregator](#). For practical proof of feasibility, please proceed to the A3 Working Prototype Demonstration.